Laporan Praktikum

Pemrograman Beriorientasi Objek

“Packages, Encapsulation, & Interfaces”



Nama: Margareth Lumban Gaol

NIM : 41425008

Prodi : D4 Teknologi Rekayasa Perangkat Lunak

INSTITUT TEKNOLOGI DEL

FAKULTAS VOKASI

1. Lakukan analisa mengapa atribut jlhTorsi, faktorPembagi, panjangGagang, dan rasioGigi termasuk setter-getter nya pada kelas Motor dan Mobil tidak dibuat di dalam IKenderaan?

   Jawaban:

   Pada program ini terdapat beberapa atribut yang digunakan oleh kelas Mobil dan Motor, yaitu jlhTorsi, faktorPembagi, panjangGagang, dan rasioGigi. Atribut tersebut tidak diletakkan di dalam IKendaraan karena atribut tersebut tidak merupakan informasi yang harus dimiliki oleh semua jenis kendaraan.

   Pada IKendaraan, method yang tersedia merupakan bagian yang bersifat umum untuk kendaraan, seperti mengatur jumlah roda, dimensi kendaraan, jumlah spion, warna kendaraan, mendapatkan informasi kendaraan, dan menghitung percepatan.

   Contohnya pada IKendaraan terdapat:

   void setJlhRoda(int value);

   void setDimensiKendaraan(int value);

   void setJlhSpion(int value);

   void setWarnaKendaraan(String value);

   Atribut seperti jumlah roda, dimensi, jumlah spion, dan warna dapat digunakan oleh berbagai jenis kendaraan, sehingga sesuai untuk menjadi bagian dari IKendaraan dan kemudian diimplementasikan oleh kelas Kendaraan.

   Sementara itu, pada kelas Mobil terdapat atribut khusus:

   private int jlhTorsi, faktorPembagi;

   private float panjangGagang, rasioGigi;

   Atribut tersebut digunakan untuk melakukan perhitungan percepatan pada Mobil. Tidak semua kendaraan membutuhkan atribut tersebut dengan cara yang sama. Contohnya, Sepeda pada program ini tidak menggunakan jlhTorsi dan faktorPembagi seperti Mobil. Jika atribut khusus tersebut dimasukkan ke dalam IKendaraan, maka setiap kelas kendaraan yang mengimplementasikan interface tersebut akan memiliki aturan atau data yang sebenarnya belum tentu dibutuhkan.

   Kesimpulan:

   Berdasarkan analisis tersebut, atribut jlhTorsi, faktorPembagi, panjangGagang, dan rasioGigi lebih tepat diletakkan pada kelas Mobil atau Motor karena atribut tersebut merupakan atribut khusus yang digunakan untuk kebutuhan kendaraan tertentu. Sedangkan IKendaraan sebaiknya hanya berisi bagian yang bersifat umum dan dapat diterapkan pada berbagai jenis kendaraan. Dengan cara ini, struktur program menjadi lebih jelas dan setiap kelas hanya memiliki atribut yang memang dibutuhkannya.

2. Ubahlah seluruh *access modifier* metode yang ada pada kelas "Kendaraan" menjadi "protected". Lakukan observasi kecil, mengapa perubahan ini mengakibatkan *error* pada program.

   Jawaban:

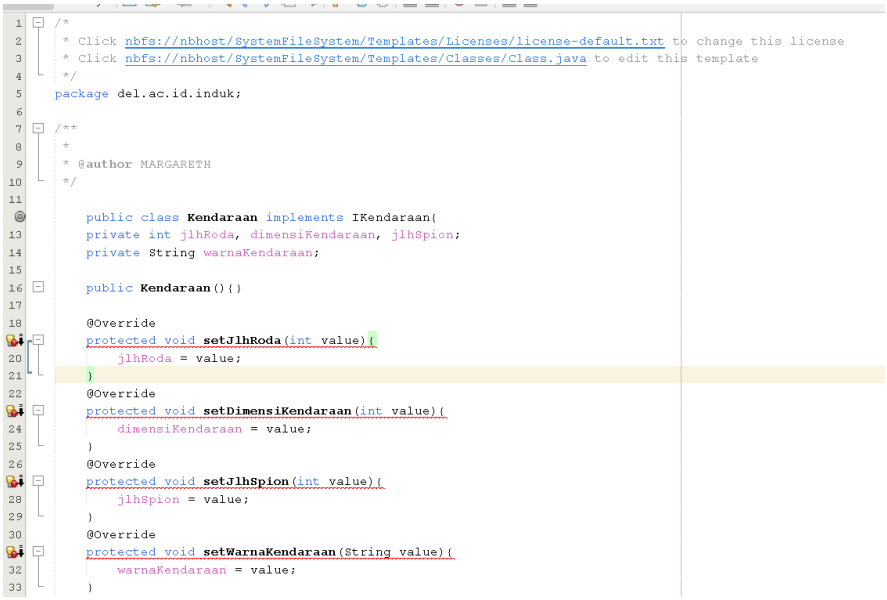
   Pada percobaan kedua, seluruh method pada kelas Kendaraan yang sebelumnya menggunakan access modifier public diubah menjadi protected.

   Contohnya, sebelum diubah:

   @Override

```
public void setJlhRoda(int value){
   jlhRoda = value;
}
```
Kemudian diubah menjadi:
```
@Override
protected void setJlhRoda(int value){
   jlhRoda = value;
}
```
Perubahan yang sama dilakukan pada method lainnya seperti setDimensiKendaraan(), setJlhSpion(), setWarnaKendaraan(), getJlhRoda(), getDimensiKendaraan(), getJlhSpion(), getWarnaKendaraan(), cetakInformasi(), dan kalkulasiPercepatan().
Setelah access modifier diubah dari public menjadi protected, program mengalami error

pada method-method tersebut:
Hal ini terjadi karena kelas Kendaraan menggunakan:
```
public class Kendaraan implements IKendaraan
```
Artinya, Kendaraan mengimplementasikan interface IKendaraan.
Pada interface IKendaraan, method seperti:
```
void setJlhRoda(int value);
```
secara otomatis memiliki access modifier public, walaupun keyword public tidak dituliskan.
Dengan demikian, method tersebut sebenarnya dianggap:
```
public void setJlhRoda(int value);
```
Ketika di kelas Kendaraan method tersebut diubah menjadi:
```
protected void setJlhRoda(int value);
```
terjadi masalah karena protected memiliki akses yang lebih terbatas daripada public.
IKendaraan menentukan method dengan akses public.
Kemudian Kendaraan harus menyediakan implementasi method tersebut dengan akses public juga. Kendaraan tidak boleh mengurangi tingkat aksesnya menjadi protected.
Kesimpulan:
Perubahan access modifier method pada kelas Kendaraan dari public menjadi protected menyebabkan error karena Kendaraan merupakan kelas yang mengimplementasikan IKendaraan. Semua method yang berasal dari interface harus tetap memiliki access modifier public ketika diimplementasikan oleh Kendaraan. Oleh karena itu, method tersebut tidak dapat diubah menjadi protected.



```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package del.ac.id.induk;

/**
 *
 * @author MARGARETH
 */

    public class Kendaraan implements IKendaraan{
    private int jlhRoda, dimensiKendaraan, jlhSpion;
    private String warnaKendaraan;

    public Kendaraan(){}

    @Override
    protected void setJlhRoda(int value){
        jlhRoda = value;
    }
    @Override
    protected void setDimensiKendaraan(int value){
        dimensiKendaraan = value;
    }
    @Override
    protected void setJlhSpion(int value){
        jlhSpion = value;
    }
    @Override
    protected void setWarnaKendaraan(String value){
        warnaKendaraan = value;
    }
```

3. Sama seperti soal no.2, lakukan perubahan *access modifier* metode pada kelas "Mobil" menjadi "protected". Lakukan observasi kecil, mengapa perubahan ini mengakibatkan *error* pada program

   Jawaban:

   Pada percobaan selanjutnya, access modifier method yang terdapat pada kelas Mobil diubah dari public menjadi protected.

   Pada kelas Mobil terdapat beberapa method, yaitu:

   setJlhTorsi()

   setPanjangGagang()

   setRasioGigi1()

   setFaktorPembagi()

   kalkulasiPercepatan()

   cetakInformasi()

   Namun, tidak semua method tersebut memiliki hubungan yang sama dengan kelas Kendaraan.

   Pada method:

   setJlhTorsi()

   setPanjangGagang()

   setRasioGigi1()

   setFaktorPembagi()

   method tersebut merupakan method khusus milik Mobil dan tidak menggunakan **@Override**.

   Contohnya:

   ```
   public void setJlhTorsi(int value){
       jlhTorsi = value;
   }
   ```

   Method tersebut secara deklarasi dapat diubah menjadi:

   ```
   protected void setJlhTorsi(int value){
       jlhTorsi = value;
   ```

}
Namun, jika method tersebut dipanggil dari class lain yang berada di package berbeda dan bukan merupakan subclass, maka method tersebut tidak dapat diakses secara langsung.

Method yang Menggunakan **@Override**
Pada Mobil terdapat dua method yang melakukan override terhadap method pada Kendaraan, yaitu:
@Override
public float kalkulasiPercepatan(...)
dan:
@Override
public void cetakInformasi(Object o)
Kedua method tersebut tidak dapat diubah begitu saja menjadi protected.
Hal ini karena pada kelas Kendaraan, kedua method tersebut memiliki access modifier public.
Mobil mewarisi method dari Kendaraan. Ketika Mobil melakukan override, access modifier method tersebut tidak boleh dibuat lebih terbatas daripada method yang ada pada parent class.
Contohnya:
// Kendaraan
public void cetakInformasi(Object o)
kemudian:
// Mobil
protected void cetakInformasi(Object o)
akan menghasilkan error.
Hal yang sama berlaku untuk kalkulasiPercepatan().
Penyebab Error:
Penyebabnya adalah Mobil mencoba mengubah access modifier dari public menjadi protected pada method yang di-override.
public memiliki tingkat akses yang lebih luas dibandingkan protected. Oleh karena itu, method hasil override tidak boleh mengurangi hak akses yang sudah diberikan oleh parent class.
Kesimpulan:
Perubahan access modifier pada kelas Mobil menjadi protected harus memperhatikan apakah method tersebut merupakan method hasil override atau bukan. Method khusus seperti setJlhTorsi(), setPanjangGagang(), setRasioGigi1(), dan setFaktorPembagi() dapat dibuat protected, tetapi aksesnya menjadi terbatas. Sementara itu, method kalkulasiPercepatan() dan cetakInformasi() yang melakukan override terhadap method public pada Kendaraan tidak dapat diubah menjadi protected karena akan mempersempit access modifier dari method parent.

Jadi, Berdasarkan ketiga percobaan, dapat disimpulkan bahwa access modifier sangat berkaitan dengan inheritance dan interface pada Java. Atribut dan method yang bersifat umum dapat ditempatkan pada interface atau kelas induk, sedangkan atribut atau method yang bersifat khusus dapat ditempatkan pada kelas turunannya. Pada saat sebuah kelas mengimplementasikan interface, method yang diimplementasikan harus memiliki akses public. Begitu juga ketika subclass melakukan override, access modifier method tidak boleh dibuat lebih terbatas daripada method pada superclass. Dengan demikian, penggunaan public dan protected harus disesuaikan dengan hubungan antara interface, superclass, subclass, dan package agar program dapat berjalan tanpa error.

